An Analysis on the Effects of Variable Bit Precision and Dynamic Range on Power, Area and Accuracy in a 65 nm GEMM Accelerator

Kelvin A. Hau and Gumer I. Rodriguez Martinez

***Abstract*—GEMM operations are the computational backbone of many modern workloads such as modern artificial intelligence (AI). Traditionally, high-precision data formats have been used to minimize computation error at the cost of high-power consumption and large silicon area. Quantization and mixed-precision computing have been proposed as solutions to this problem. Given the surge in different data formats, this project aims to conduct a systematic analysis on the effects of multiple bit width configurations alongside different numerical representations on power, area, and accuracy on a 65 nm GEMM unit implemented using a systolic array architecture. The results indicate that a significant reduction in data precision can lead to a substantial increase in throughput, lower area footprint, and lower power consumption. Our findings are summarized in comparative tables and visualizations that can provide insights into resource requirements of GEMM accelerators at early stages of the design.**

I. Section I: Overall IDEA

GENERAL Matrix Multiplication (GEMM) operations constitute the computational backbone of many modern workloads such as Machine Learning (ML). Historically, these workloads were tackled using high-precision floating-point numbers to maximize dynamic range and minimize error due to computation accuracy. However, this results in GEMM accelerators with large area overheads and large power consumption. Recent research has shown that lower-precision formats can be used in modern ML workloads with minimal impact on model accuracy, giving rise to quantization techniques and mixed-precision computing [1].

Given the need for many different data formats, this project aims to perform a quantitative and comparative analysis of the trade-offs in this space, with a focus on design key performance indicators (KPIs) such as throughput, power consumption, and silicon area across multiple configurations of bit width and numerical representation (fixed point versus floating point) within a GEMM unit implemented with a systolic array architecture. The design will be implemented in TSMC 65 nm CMOS technology, with the analysis restricted to synthesis results. Furthermore, the project seeks to characterize and visualize the inherent trade-offs among these KPIs, with particular emphasis on their implications for dynamic range and accuracy.

Even though the main motivation for this project comes from the use of GEMM accelerators in machine learning and modern neural network architectures, we will not evaluate the impact of bit precision on the results of these models. Instead, we constrain the scope of this project to the results of random matrix multiplications.

The analysis will consider the following configurations:

- Floating-point: 32-bit, 16-bit, and 8-bit
- Integer: 32-bit, 16-bit, and 8-bit

II. Section – 2: Background

A. Modern AI

Nowadays, artificial intelligence (AI) is used in many applications, such as data analytics, computer vision, language processing, and robotics. Deep neural networks (DNNs) are at the core of these models, and their inference accuracy comes at the cost of high computational complexity and large storage requirements [2].

Scaling these DNNs to run on hardware-constrained edge devices has become increasingly important. The networks must be accurate enough without overusing hardware resources, so they need to be efficient and meet strict processing-time requirements [2].

Extensive research has been done to achieve increasingly efficient designs, both at the model level and in hardware. These efforts include parameter search, sparsification, quantization, and digital acceleration [2].

Transformers, a specific type of DNN, have gained significant popularity in today's AI models. Due to their requirements on large memory and computational resources, they rely on efficient matrix multiplication accelerators [3].

B. Systolic Arrays

AI workloads suffer from memory and data bottlenecks, so spatial architectures such as systolic arrays have emerged as the go-to approach to address this inherent problem. Systolic arrays are formed by a two-dimensional array of processing elements (PEs), each performing one or more arithmetic operations, such as multiplication and accumulation. Data propagates across the array in a rhythmic manner, mimicking a pulse—hence the name. The main goal is to maximize the number of operations per memory read by reusing data among the PEs and thereby reducing memory accesses. This architecture is especially beneficial for deep learning workloads, particularly for matrix multiplications. [3]

C. Arithmetic precision

As a rule of thumb, if you halve the precision of an arithmetic

operation, the time it takes to perform that operation is also reduced by half. It also consumes half the storage and half the communication cost. Whether a given precision is sufficient depends on the specific application, and even within the same field, different computations may require different minimum precisions. In modern hardware, lower-precision arithmetic has become more attractive due to its advantages in speed, energy, storage, and transfer costs. This leads to a mixed-precision world, where precision can be chosen to make hardware more efficient by optimizing computational resources [4].

In recent years, machine learning has increasingly adopted low-precision arithmetic because some parts of the computation can be executed with reduced precision without a noticeable impact on result quality, while benefiting from the advantages of lower precision [4].

D. *State of the art*

Several AI accelerators in use today support mixed precision, such as Google TPUs (v2, v3, v4i), NVIDIA Tensor Cores (V100, A100, T4), and ARMv8.6-A. They use different matrix dimensions ranging from 128 down to 8, 4, or even 2. However, details of their implementations are generally not publicly available [4].

In [5], the authors investigate the trade-offs between accuracy and throughput for neural networks using reduced precision on reconfigurable logic, including computational resources and memory usage in the analysis.

E. *This work*

In this work, we perform a quantitative analysis of the trade-offs in throughput, power, silicon area, dynamic range, and accuracy of a GEMM accelerator using a systolic array architecture across multiple configurations of bit width and numerical representation. Our design is implemented in TSMC 65 nm CMOS technology, with the analysis restricted to synthesis results obtained using Synopsys Design Compiler. Although the results do not come from an actual layout, they can be used as a proxy, and the ratios between configurations should not change significantly, given the regular structure of systolic arrays. To the best of the authors' knowledge, this type of analysis for a given technology node is rarely found in the literature.

The main contribution of this work is the development of comparative tables and visualizations that clearly highlight the expected trade-offs among different configurations of bit width and numerical representation. Such representations are particularly useful in the early stages of a project, as they provide insight into the resource requirements associated with specific precision levels and enable more informed design decisions.

III. CIRCUIT IMPLEMENTATION

A. *RTL implementation*

For our GEMM accelerator, we used a weight-stationary systolic array architecture with parameterizable dimensions (and thus number of processing elements), bit widths, and data formats (integer versus floating point). The systolic array contains a grid of PEs, input and output buffers, and a controller to read from memories and write the results to another memory. Each PE contains a multiply–accumulate (MAC) circuit plus a register. The design was implemented in SystemVerilog.

We also created a Python script to generate the high-level behavior of the matrix multiplier, with bit-level precision, given the chosen parameters. Random input matrices and the corresponding result matrices are generated, and these input matrices are used to feed our GEMM implementation in a SystemVerilog testbench. The outputs are then compared against the reference results to verify the correctness of the RTL implementation.

Since our computer hardware supports 64-bit and 32-bit floating point arithmetic, the results of the Python simulation and RTL matched perfectly, bit for bit, for these precisions. However, because there is no native hardware support for 16-bit and 8-bit floating point, the simulations for these formats exhibit small numerical differences. Nevertheless, we can reasonably assume that the RTL implementation for low precision is correct for two reasons: (i) the errors are small, even though Python internally uses higher precision; and (ii) the design is fully parameterizable, meaning that the structure for 16- and 8-bit configurations is the same as for 32- and 64-bit, so the likelihood of a functional bug is low unless it is due to an edge case.

Once the RTL design was functionally verified, it was synthesized using Synopsys Design Compiler (DC) targeting TSMC 65 nm CMOS technology, and the resulting netlist was verified in the same way as the SystemVerilog code.

B. *Selected configurations*

Once the RTL and synthesis flow were completed, we selected specific configurations for the analysis. The systolic array dimensions were fixed to 8 × 8, and we conducted experiments with the following formats:

- Floating-point: 32-bit, 16-bit, and 8-bit
- Integer: 32-bit, 16-bit, and 8-bit

For 32-bit and 16-bit floating point, we used IEEE 754 formats with 1 sign bit, 8 exponent bits, and 23 mantissa bits, and 1 sign bit, 5 exponent bits, and 10 mantissa bits, respectively. For 8-bit floating point, we selected 1 sign bit, 4 exponent bits, and 3 mantissa bits, a commonly used format for deep learning [6].

Table 1 summarizes the dynamic range of the different formats. Min is the smallest positive nonzero normal value. Max is the largest normal value. Range is the difference between the highest and lowest values.

Each configuration includes 2kB of scratch memory that is divided into 4 distinct buffers used for inputs, outputs, and intermediate computations. It is worth noting that each buffer has port widths that adapt alongside the dimensions of the array. For example, an 8x8 8-bit configuration contains four buffers of size 4096 (512B), each buffer requires a port width of 8x8 (8 PEs x 8 bits), resulting in a memory depth of 64. On the other hand, an 8x8 32-bit configuration has a lower memory depth of 16 to account for the larger port width.

Table 1. Details on formats.

Format	Min	Max	Range
32-bit fp	1.18 x 10 ^ -38	3.40 x 10 ^ 38	6.80 x 10 ^ 38
16-bit fp	6.10 x 10 ^ -5	6.55 x 10 ^ 4	1.31 x 10 ^ 5
8-bit fp	1.56 x 10 ^ -2	4.48 x 10 ^2	8.96 x 10 ^2
32-bit int	1	2.147 x 10 ^ 9	2^32
16-bit int	1	3.27 x 10 ^4	2^16
8-bit int	1	1.27 x 10 ^ 2	2^8

IV. EXPERIMENT RESULTS

A. *Performance, Power, and Area (DC Results)*

Table 2 presents the Area, Power, and Target Period results obtained from Synopsys Design Compiler (DC) for all tested formats (8-bit, 16-bit, and 32-bit floating-point and integer). The target period is defined as the inverse of the maximum operating frequency and serves as a primary proxy for performance (further discussed in Section IV-C).

Table 2. Main KPIs extracted from synthesis reports

Format	*Area (mm^2)	**Power (mW)	Target Period (ns)
32-bit fp	1.42	43.38	3.00
16-bit fp	0.763	26.10	2.50
8-bit fp	0.478	19.08	2.00
32-bit int	0.977	44.18	2.00
16-bit int	0.443	22.18	1.50
8-bit int	0.314	19.43	1.00

**Area measurements are restricted to the total cell area. No placement or routing is considered for area calculations during Synthesis.*

***Power measurements are restricted to the summation of combinational power and sequential power. Clock network power is not accounted for due to the lack of a clock distribution during Synthesis.*

Figure 1 illustrates the Key Performance Indicators (KPIs) from Table 2. The results align with the prediction that higher bit widths require increased silicon area, power, and latency. Notably, the power consumption for some integer formats is higher than that of their floating-point counterparts. This is attributed to the higher operating frequencies (lower target periods) achievable by the integer designs, given the direct relationship between dynamic power and frequency. We discuss these specific power findings further in Section VI.

The critical comparison lies between floating-point and integer configurations of the same bit width. Floating-point configurations incur area overheads of 45.3%, 72.2%, and 52.2% for 32-bit, 16-bit, and 8-bit widths, respectively, resulting in an average area penalty of 56.6%. Furthermore, floating-point support introduces a consistent 1 ns latency penalty across all widths. Consequently, the data confirms that hardware support for floating-point formats incurs substantial area, power, and latency overheads.

B. *Floating-Point Computation Accuracy*

To evaluate the accuracy of the floating-point configurations, we developed an automated test suite combining Python and SystemVerilog. Each test processes 1,000 randomized input matrices. We calculated the error as the Root Mean Squared Relative Error (RMSRE) between the hardware output and a 64-bit floating-point golden reference generated in NumPy. Figure 2 summarizes these error results for each configuration against the baseline.

Figure 1. KPI metrics vs. bit width for Floating Point (solid) and Integer (dashed) formats

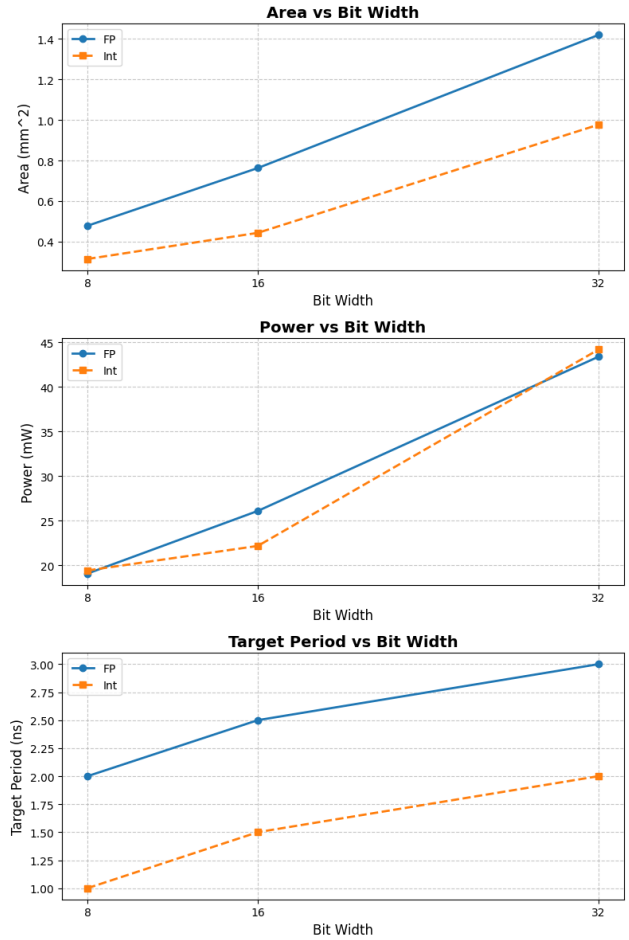
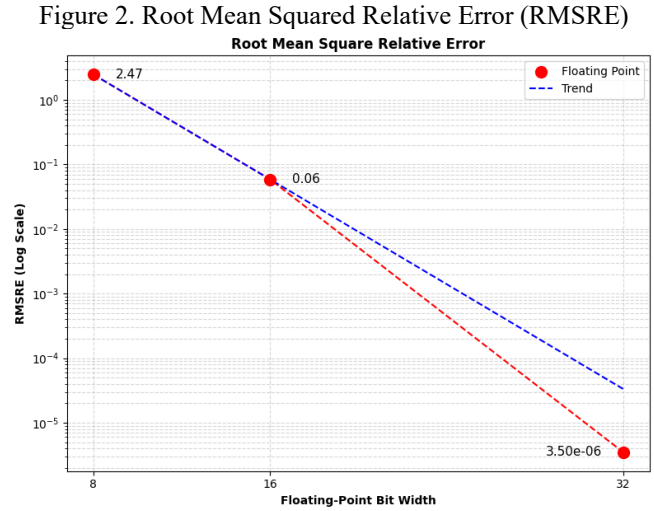


Figure 2. Root Mean Squared Relative Error (RMSRE)



As expected, we observe a large exponential relationship in the measured error and the floating-point bit width.

C. *Performance Metric: Throughput*

Our primary performance KPI is throughput, defined as the quantity of bits processed per unit of time. To estimate throughput within our experimental setup, we rely on two simplifying assumptions. First, we assume flexible memory

port widths, implying that buffers provide sufficient bandwidth (ranging from 64 to 256 bits per cycle) to prevent memory bottlenecks. Second, we standardize the workload, assuming all configurations compute 8 x 8 matrix multiplications. While the dimensions are fixed, our model accounts for variations in bit widths and value ranges.

Based on these assumptions, we approximate throughput for each configuration using the following expression:

$$Throughput = \frac{\#bits}{N \times T}$$

Where #bits correspond to the total computed bits, N is the total number of cycles per computation (32 cycles), and T is the clock period. Table 3 summarizes the throughput results.

Table 3. Throughput results for each HW configuration

Format	Throughput (Gbps)
32-bit fp	21.33
16-bit fp	12.80
8-bit fp	8.00
32-bit int	32.00
16-bit int	21.33
8-bit int	16.00

D. IEE 754 Compliance

Our next experiment compares the overhead of Nan and Denormal hardware support. Table 4 summarizes the KPI results of a 32-bit floating-point configuration with and without IEEE 754 compliance.

Table 4. IEEE 754 compliance comparison table

Format	IEEE 754 Compliance	Area (mm^2)	Power (mW)	Target Period (ns)
32-bit fp	Yes	1.42	43.38	3.00
32-bit fp	No	1.22	36.15	3.00

As expected, there is a 16.39% area overhead, and 20% power overhead when enabling IEEE 754 compliance for 32-bit floating-point. These results support the idea that IEEE 754 compliance comes at the cost of area, and power, due to additional hardware requirements. Latency overhead was not explored in this experiment.

V. Collaboration Plan

Gumer was responsible for developing the baseline, parameterizable RTL implementation of the systolic array in integer format. Kelvin extended the design to support floating-point configurations and verified the correctness of both formats using a SystemVerilog testbench and Python scripts to produce the expected results. The synthesis flow and the analysis of the resulting data were set up and conducted jointly by Gumer and Kelvin.

In preparing this report, Gumer developed the overall structure and drafted the main content, while Kelvin extracted the KPIs for each scenario, generated the plots, and finalized the manuscript.

Tasks were not intended to be strictly divided; support was provided to one another. Collaboration was facilitated through a shared Git repository, and feedback was actively exchanged between the team members as needed.

VI. Conclusion and Future Work

As expected, the floating-point implementations provide superior dynamic range but incur larger silicon area and lower throughput compared to the integer implementations. Power consumption is similar for both formats; however, because they operate at different frequencies, the floating-point implementation consumes more energy per operation, since the integer implementation completes the same workload more quickly. It is also worth noting that making the floating-point implementation compliant with the IEEE 754 standard has a non-negligible cost in both area and power. With the generated tables and plots, it becomes easier to visualize these trade-offs in critical KPIs, enabling more informed design decisions at the beginning of a project.

A primary limitation of this study is the exclusion of physical design data. Preliminary tests indicate that invoking an Auto Place & Route (APR) flow provides significantly increased area, power, and latency results. Consequently, incorporating a full APR flow for a future revision is essential for improving the fidelity of the extracted KPIs. Furthermore, dynamic power measurements are directly correlated with workload activity. An extension of this work would involve re-evaluating power consumption using activity traces from realistic workloads, such as a convolutional layer of a neural network.

VII. References

[1] Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R., & Bengio, Y. (2018). Quantized neural networks: Training neural networks with low precision weights and activations. journal of machine learning research, 18(187), 1-30.

[2] A. N. Mazumder et al., "A Survey on the Optimization of Neural Network Accelerators for Micro-AI On-Device Inference," in IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 11, no. 4, pp. 532-547, Dec. 2021, doi: 10.1109/JETCAS.2021.3129415.

[3] A. J. Abdelmaksoud, C. Sestito, S. Wang, and T. Prodromakis, “ADiP: Adaptive Precision Systolic Array for Matrix Multiplication Acceleration,” arXiv preprint arXiv:2510.10623, 2025. Available: https://arxiv.org/abs/2510.10623

[4] Higham NJ, Mary T. Mixed precision algorithms in numerical linear algebra. *Acta Numerica*. 2022;31:347-414. doi:10.1017/S0962492922000022

[5] J. Su, N. J. Fraser, G. Gambardella, M. Blott, G. Durelli, D. B. Thomas, P. Leong, and P. Y. K. Cheung, “Accuracy to Throughput Trade-offs for Reduced Precision Neural Networks on Reconfigurable Logic,” arXiv preprint arXiv:1807.10577, 2018. Available: https://arxiv.org/abs/1807.10577

[6] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu, “FP8 Formats for Deep Learning,” arXiv preprint arXiv:2209.05433, 2022. Available: https://arxiv.org/abs/2209.05433